

C Plus Plus C++

file extension → .cpp

basic code → #include <iostream> → header file
(helps in input and output)
using namespace std;
int main() { cout << "Hello"; → like a batch.
return 0; }

:: scope resolution operator

Variables → data storage containers. eg → int sum = 6;

Comments → single line → //

multiline → /* */

* double stores more decimal places
as compared to float.

Scope → region where a variable exists

Types → ① local → within a function

② global → can be accessed anywhere in the program
declared outside a function.

global and local variables can have the same name in C++.

Datatypes → ① Built-in ② User defined ③ derived.

Built in → ① int ② float ③ char ④ double ⑤ Bool.

user defined → ① Struct ② union ③ enum

derived → ① array ② functions ③ pointers

local variables take precedence over global variables.

The variable names in C++ can range from 1 to 255 characters.
All the variable naming rules are the same as C language.

Basic Input and output in C++

istream → input output stream

Stream → Sequence of bytes corresponding to input and output.

input stream → direction of flow of bytes from input to the main memory.

output stream → from main memory to output devices.

<< is called insertion operator.

>> extraction operator.

Syntax to take input

→ this a simple addition program.

```
#include <iostream>
using namespace std;

int main() { int num1, num2;
cout << "Enter the values of num1 and num2:";
cin >> num1; cin >> num2;
cout << "The Sum is:" << num1 + num2;
return 0; }
```

Keywords → reserved by the language.

refer online documents for the list.

Header files

There are 2 types of header files

- ① System header files → Come with the compiler
- ② Userdefined header files → written by the programmer.

① Syntax → #include <iostream>

② Syntax → #include "filename.h"

note → this file should already be included in the folder. you're working with.

(for reference use <http://preference.com>)

Operators in C++

Types → ① Arithmetic operators → $+$, $-$, $*$, $/$, $\%$ ^{modulo}
 $a++$, $a--$, $++a$, $--a$

② Assignment operators → $=$, $+=$, $-=$, $*=$, $/=$

③ Comparison operators → $<$, $>$, $=$, $<=$, $>=$, $!=$

← note → while writing them, keep the operands in parentheses.

Eg → `cout << "The value of a == b is: " << (a == b) << endl;`

True means 1
False means 0

 and or not operator.

④ Logical operators → $\&\&$, $\|\|$, $!$

$::$ → Scope resolution operator

if you write ~~variable~~ $<< ::$ variable name.

Then the value of the global variable will be printed even if a local variable with the same name exists.

By default, the decimal value containing variables are considered of the double data type by the compiler.

to pass as floating point number write eg → `34.4f`

for comparison operators.

34.4 $\rightarrow$ double 34.4f $\rightarrow$ float] $\rightarrow$ difference is seen at the time of function overloading.

[f or F both work]

sizeof(34.4);

output $\rightarrow$ 8

Reference Variables

```
int x = 45;
```

```
int &y = x;
```

```
cout << x << endl;  $\rightarrow$  output  $\rightarrow$  45.
```

```
cout << y << endl;  $\rightarrow$  output  $\rightarrow$  45.
```

Type Casting

```
int a = 45;
```

```
cout << a;  $\rightarrow$  int
```

```
cout << (float)a;  $\rightarrow$  float
```

[(float)a = float(a)] $\rightarrow$ They both mean the same.

Constants $\rightarrow$ their values can't be changed.

```
const int a = 3;
```

Keyword

Manipulators

Help in code formatting.

① `endl;` → takes the cursor to the next line.

② `#include <iomanip>`

`int a = 3, b = 78, c = 12;`

`cout << "The value of a is : " << setw(4) << a << endl;`

character spacing.
helps in setting the width (4 character length)

output → The value of a is : 3

4 character space.

Operator Precedence and associativity.

highest precedence is of `::` Scope resolution operator.

refer C++ reference.

Flow Control

Basic Control Structures

- ① Sequence Structures
- ② Selection Structure
- ③ Loop Structure.

Sequence Structure $\rightarrow$ line by line flow of control.

Selection Structure $\rightarrow$ ——— depending on the choices. (if/else).

loop Structure $\rightarrow$ program keeps running ~~until~~ ^{till} the condition gets fulfilled.

if else statements.

```
int i = 0;
```

```
if (i < 3) { cout << i; return 0; }
```

if-else ladder $\rightarrow$ else if

Switch Case.

Switch case syntax $\rightarrow$

```
Switch (expression) {
```

```
    Case 1:
```

```
    {
```

```
        // code; break;
```

```
    Case 2:
```

```
    {
```

```
        // code; break;
```

```
    default:
```

```
    {
```

```
        // code;
```

```
    }
```

Loops

for (initialisation; condition; updation)

① For loop → Syntax → `for (int i=0; i<100; i++)`
`{ //code }`

`int i=0;`

② while loop → Syntax → `while (i<100)` `(i<100)` `{`
`//code }`
`i++`

③ do-while loop → Syntax → `do { //code; i++; }` `while (i<100);`

← This is very important

break and continue

break → helps breakout of a loop / function or a switch case statement.

Continue → skips the iteration written after the continue keyword.

Pointers

Pointer is a datatype which holds the address of other datatypes.

```
int a = 3;
```

```
int *b = &a;
```

& → address of operator

* → dereference operator.

```
cout << "address of a = " << &a << endl;
      << b << endl;
```

→ prints the address of a.

```
cout << "The value at the address stored in b = " << *b << endl;
```

prints the value at that address stored at b.

```
int **c = &b; → pointer to pointer.
```

Arrays

Homogenous data collection stored in a contiguous memory location.

Syntax → `int marks [size of array] = {23, 99, 49, 98};`

```
or int engmarks[4];
```

```
engmarks[0] = 22 << endl;
```

```
_____ [1] = 49 _____;
```

```
_____ [2] = 47 _____;
```

printing array elements using a loop $\rightarrow$

```
for (int i = 0; i < 4; i++)
```

```
{
```

```
    cout << marks[i] << endl; }
```

```
int i = 0;
```

```
while (i < 4) {
```

```
    cout << marks[i] << endl;
```

```
    i++; }
```

```
int i = 0;
```

```
do { cout << marks[i] << endl;
```

```
    i++; } while (i < 4);
```

Pointers and Arrays.

In arrays, the name of the array is the address of the first block of the arrays.

& array name $\rightarrow$ this is wrong.

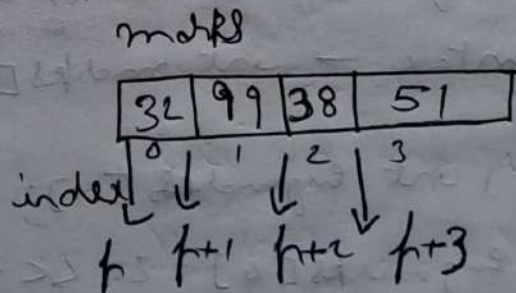
~~$p = \text{marks}[0]$~~ $*p = \text{marks};$

~~$p++$~~ $x(p) = 32$

$*(p++) = 99$

$*(p+2) = 38$

$*(p+3) = 51.$



pointer arithmetic formula

$$\begin{array}{ccccc} \text{address}_{\text{new}} & = & \text{address}_{\text{current}} & + & i * \text{sizeof}(\text{data type}) \\ \downarrow & & \downarrow & & \\ (p+i) & & p & & \end{array}$$

eg → for integer (byte size 4)

$$\begin{aligned} \text{new address} &= 32 + 1 \times 4 \\ &= 36 \end{aligned}$$

Code example → `int marks[] = { 4, 8, 10, 12 };`

`*p = marks;`

`marks[0]`

`cout << "The value at p:" << *p << endl;`

`marks[1] << *(p+1) << endl;`

`marks[2] << *(p+2) << endl;`

`marks[3] << *(p+3) << endl;`

Output → The value at marks[0]: 4

The value at marks[1]: 8

The value at marks[2]: 10

The value at marks[3]: 12

Structures, Union and Enum.

Syntax → `// before int main() { }`

`struct employee`

`{ int id;`

`float salary; }`

```
int main() {
    struct employee a1; → creating an entity
    a1.eId = 1; → giving value
    a1.salary = 1200000; → accessing value.
    cout << "The eId of a1 is:" << a1.eId << endl;
    return 0; }
```

other way → typedef struct employee
{
 } ef;

```
int main() {
    ef a1;
    return 0; }
```

→ What happened is
just like in python
(import pandas as pd)

you can give alias ^{name} ~~names~~
to employee so that you
don't always have to
write ~~struct~~

struct employee employee

name.

you can shorten it
like ef employee-name

← ef is just like a
name you can use any
other name as well.

Unions

They provide better memory management.

Syntax → union money

4 bytes ← int rice;
1 byte ← char car;
4 bytes ← float dollar; };

[The maximum number of memory bytes will be allocated, instead of all i.e., instead of $4+1+4$ bytes only $= 9$ 4 bytes will be allocated]

→ you can use only one liberty to use anyone of the. So, you can use union.

you can use any one at a time, hence, you get better memory management with union.

[The memory gets shared]

Rest of the syntax is the same as that of a Structure.

Enum

Increases the program readability.

① int main() {
enum Meal { breakfast, lunch, dinner };

cout << breakfast;

cout << lunch;

cout << dinner;

output ⇒ 0 1 2

② Meal m1 = lunch;

cout << m1;

output ⇒ 1

Functions and function prototypes.

Syntax $\rightarrow$

```
int sum(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

```
int main() {  
    int m1, m2;  
    cout << "Enter 1st number: " << endl;  
    cin >> m1;  
    cout << "Enter 2nd number: " << endl;  
    cin >> m2;  
    cout << "The sum is: " << sum(m1, m2);  
    return 0;  
}
```

$\rightarrow$ function call.

Function prototyping $\rightarrow$

```
datatype function name (arguments);
```

Eg $\rightarrow$

```
int sum(int a, int b);
```

function definition $\rightarrow$

```
int sum(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

```
int sum(int, int);
```

This is correct

```
int sum(int a, b);
```

This is wrong.

Formal parameters $\rightarrow$ parameters or arguments that we make in the function like a and b .

Actual parameters $\rightarrow$ Values that are passed into the function like m_1 and m_2 .

`Void g(Void) {`
`cout << "good morning"; }`
or `Void g()  $\rightarrow$`

They both are the same and they return nothing.

Call by Value and Call by reference

Call by Value $\rightarrow$ The previous Sum function is the perfect example.

Call by reference $\rightarrow$ `Void Swap(int a, int b) {`
`int t = a;`
`a = b;`
`b = t; }`

```
int main() {  
    int x=4, y=5;  
    cout << x << y;  
    Swap(x, y);  
    cout << x << y;  
}
```

Output $\rightarrow$

4 5
4 5

$\rightarrow$ because only a copy of the actual arguments gets passed.

```

Void SwapPointer (int *a, int *b) {
    int t = *a;
    *b = a;
    *a = *b;
    *b = t; }

```

output → 45
54

```

int main() {
    int x = 4, y = 5;
    cout << x << y;
    SwapPointer (&x, &y);
    cout << x << y;
    return 0; }

```

→ This will work because it is working using the memory addresses.

Call by reference by using C++ reference variables.

```

SwapRefVar (int &a, int &b) {
    int t = a;
    a = b;
    b = t; }

```

```

int main() { int xy = 4, yx = 5;

```

```

    cout << x << y;

```

```

    SwapRefVar (x, y);

```

```

    cout << x << y;

```

```

    return 0; }

```

Return a reference
you can look up for it online, if needed.

[it is a request
(compiler can
deny it)]

Inline Functions

→ This code gets directly replaced in the
function call.
(compile time)

eg → `inline int product(int a, int b) {
 return a * b; }`

If we use inline functions with functions performing
very big lines of code, the cache memory will get
filled up and it will cause problems.

do not use inline functions with ~~recursion~~ ^{recursion} and static
variables.

Static variables → `int product(int a, int b) {
 static int c = 0; → This executes only once
 return c * c + 1;
 return a * b + c; }`

←
next time this
function is run, the
value of c will be
retained.

Default arguments

→ arguments with default values need to be
added later.

`int moneyreceived(int current, int factor = 1.04) { }`
↳ default value.

Constant arguments

`int strlen(const *p) { }`
↳ this value can't be changed.

Recursion and Recursive functions.

$$0! = 1 \quad 1! = 1$$

$$n! = n * (n-1)!$$

```
#include <iostream>
using namespace std;
int main() {
```

```
    return 0; }
```

```
int factorial(int n) {
```

```
    return
```

```
    if ((n == 0) || (n == 1)) {
```

```
        return 1; }
```

```
    return n * factorial(n-1); }
```

Function Overloading

```
int sum(int a, int b) {
    return a + b; }
```

```
int sum(int a, int b, int c) {
    return a + b + c; }
```

```
int main() {
    cout << sum(3, 6);
    cout << sum(3, 6, 7);
    return 0; }
```

output $\rightarrow$ 9 16

C++ function overloading ^{available} smartly uses the best ~~needed~~ function with the help of matching arguments.

Object Oriented Programming

A class is a template to create objects.

Terminology →

- ① Class → Basic template for creating objects.
- ② Objects → Basic runtime entities.
- ③ Data Abstraction and encapsulation → wrapping data and functions into a single unit (class).
- ④ Inheritance → properties of one class can be inherited into others.
- ⑤ Polymorphism → ability to more than one form.
- ⑥ Dynamic binding → which code will run, you will know that at run time only. also known as late binding.
- ⑦ Message passing → flow of data.

Object. message (information)
↓
format.

Classes

Syntax → #include <iostream>
using namespace std;

class Employee

```
{ private:
    int a, b, c;
    public:
    int d, e;
    void setdata (int a1, int b1, int c1);
    void getdata () {
        cout << "The value of a is " << a << endl;
        _____ b _____ b _____;
        _____ c _____ c _____;
        _____ d _____ d _____;
        _____ e _____ e _____;
    }
};
```

access modifiers

to tell that access the function declared in employee class

```
void Employee :: setdata (int a1, int b1, int c1) {
    a = a1;
    b = b1;
    c = c1;
}
```

```
int main () {
```

```
    Employee harry harry;
```

```
    harry.d = 34;
```

```
    harry.e = 89;
```

```
    harry.a = 134; // This will through an error as a is private.
```

```

Harry. Setdata(1, 2, 4);
Harry. getdata();
return 0;
}

```

OPPs summary →

Class → extension of Structures.

limitations of Structures → ① public members
② no methods.

Structures in C++ can be created even without the use of typedef.

You can declare objects along with the class declaration →

Syntax → class Employee {

// class definition

} Harry, Rohan;

Harry. Salary = 8 makes no sense if Salary is private.
You will need to use a function.

Nesting of member functions
→ you can nest functions like loops.

Syntax / eg → #include <iostream>
#include <string>
using namespace std;

```
class binary {
```

```
    string s;
```

```
    public:
```

```
        void read();
```

```
        void chk_bin();
```

```
};
```

```
void binary::read() {
```

```
    cout << "Enter a binary number" << endl;
```

```
    cin >> s;
```

```
}
```

```
void binary::chk_bin() {
```

```
    for (int i = 0; i < s.length(); i++) {
```

```
        if (s.at(i) != '0' && s.at(i) != '1') {
```

```
            cout << "Incorrect binary format" << endl;
```

```
            exit(0);
```

```
        }
```

```
    }
```

If you have this function in binary then you as a user can't access them.

```
int main() {
```

```
    binary b;
```

```
    b.read();
```

```
    // b.chk_bin();
```

```
    return 0;
```

if you want this to happen or its over you can add this in the function definition of any other function.

Memory allocation

```
class Shop {
```

```
    int itemid[100];
```

```
    int itemprice[100];
```

```
    int counter;
```

```
public:
```

```
    void initcounter() { counter = 0; }
```

```
    void setprice();
```

```
    void displayprice(); }
```

```
void Shop::setprice() {
```

```
    cout << "Enter id of your item number" << counter+1 << endl;
```

```
    cin >> itemid[counter];
```

```
    cout << "Enter price of your item" << endl;
```

```
    counter++; }
```

```
void Shop::displayprice() {
```

```
    cout << "The price of item with id" << itemid[i] << " is" <<  
    itemprice[i] << endl; }
```

```
int main() {
```

```
    Shop dukaan;
```

```
    dukaan.initcounter();
```

```
    dukaan.setprice();
```

```
    dukaan.displayprice();
```

```
    return 0; }
```

Static data members & methods in C++ OOPS
Static variables need to be declared outside of the classes.

```
#include <iostream>
using namespace std;

class Employee { int id; count static int count;
public:
    void Setdata() { cin >> id; }
    void getdata() { cout << id; }
};

int Employee::count; // Default Value is 0.

int main() {
    Employee harry;
    harry.Setdata();
    harry.getdata();
}
```

count = 1000;
 syntax error.

cout << "Enter the id:" << endl;

Count = 1000;
 correct syntax.

all the objects in the class share the same count variable.

very static or class variable is a class property.

Static functions

A function that can access only the static members (variables / functions).
They can run without making any object in the class.

eg → static void getCount() {
 cout << "The value of count is:" << count << endl; }

Array of objects.

```
#include <iostream>
using namespace std;
class Employee {
```

```
    int id;
```

```
    int salary;
```

```
public:
```

```
    void setid() {
```

```
        salary = 122;
```

```
        cout << "Enter the id of employee:" << endl;
```

```
        cin >> id; }
```

```
    void getid() {
```

```
        cout << "The id of this employee is:" << id << endl; }
```

```
};
```

```
int main() {
```

```
    Employee fb[4];
```

```
    for (int i = 0; i < 4; i++) {
```

```
        fb[i].setid();
```

```
        fb[i].getid(); } return 0;
```

```
}
```

You can pass objects as function arguments.

Friend Functions in C++

```
#include <iostream>
using namespace std;
```

```
class complex {
```

```
    int a, b;
```

```
    public:
```

```
        void setnumber (int n1, int n2) { a = n1; b = n2; }
```

→ this is like a written permission for the outside function to access the private members of the class.

```
friend complex sumcomplex (complex o1, complex o2);
void printnumber () { cout << "your number is" << a << "+" << b <<
    "i" << endl; }
};
```

```
complex sumcomplex (complex o1, complex o2) {
    complex o3;
```

```
    o3.setnumber ((o1.a + o2.a), (o1.b + o2.b));
    return o3; }
```

```
int main () {
```

```
    complex c1, c2, sum;
```

```
    c1.setnumber (2, 4);
```

```
    c1.printnumber();
```

```
    c2. _____;
```

```
    c2. _____;
```

```
    sum = sumcomplex (c1, c2);
```

```
    sum.printnumber(); return 0;
};
```

Properties of friend functions →

- ① They are not in the scope of the class. They just have access to the private members of the class.
- ② as they are not in the scope of the class they can't be called from an object of that class.
eg → 1. sumComplex, it invalid.
- ③ Can be invoked without any object help.
- ④ Usually contain objects as arguments.
- ⑤ Can be declared in both either private or public part of the class.
- ⑥ Can't access the members directly by their names and need object name. member name. to access any member.

Friend classes & member friend functions

```
#include <iostream>
```

```
using namespace std;
```

class complex; → forward declaration meaning that this class is present later in the code.

```
class calculator {
```

```
public:
```

```
int add(int a, int b) {
```

```
    return(a + b); }
```

```
int SummedComplex(complex, complex); }
```

```
class complex {
```

```
    int a, b;
```

→ let calculator's SummedComplex access all your private data.

```
friend int calculator::SummedComplex(complex, complex);
```

```
public:
```

```
void SetNumber(int n1, int n2) {
```

```
    a = n1;
```

```
    b = n2; }
```

```
void PrintNumber() {
```

```
    cout << "your number is" << a << " + " << b << " i endl;
```

```
    }
```

```
};
```

```
int calculator::SummedComplex(complex o1, complex o2) {
```

```
    return(o1.a + o2.a); }
```

```
int main() {
```

```
    complex o1, o2;
```

```
    o1.SetNumber(1, 4);
```

```
    o2.SetNumber(5, 7);
```

```

Calculator calc;
int res = Calc.SummedComplex(C01, C02);
cout << "The sum of the real part of C01 and C02 is" << res << endl;
return 0; }

```

The way to make the whole calculator ^{to} ~~class~~ class your friend instead of just one function. →

Write the following lines in the complex class →
friend class calculator;

Constructors → a special member function with the same name as that of the class, used to initialize the objects of its class.
It is automatically invoked (called) ~~when~~ whenever an object is created.

eg → #include <iostream>
using namespace std;

```
class complex {
```

```
    int a, b;
```

```
    public:
```

```
    complex(); // constructor declaration.
```

```
    void printNumbers() { cout << "your number is" << a << " + " << b << "i";  
                        << endl; };
```

```
complex::complex() {  
    a = 10;  
    b = 20; }
```

```
int main() {  
    complex C1;
```

C1. print number (); // we didn't have to call the complex function - it got called automatically.
return 0; }

output $\rightarrow$ your number is 10 + 0i.

A constructor which takes no parameters is called a default constructor.

Properties of Constructors $\rightarrow$

- ① Should be declared in the ~~first~~ public section of the class.
- ② They are automatically invoked whenever an object is created.
- ③ They can't return values and don't have return types.
Not even void.
- ④ It can have default arguments.
- ⑤ We can't refer to their reference.

Default and Parameterised Constructor $\rightarrow$

$\downarrow$

This has been already discussed.

$\downarrow$

a constructor that takes parameters as arguments

The code is more or less the same only, just a few changes. $\rightarrow$

public:

Complex(int, int); };

Complex::Complex(int x, int y) {

a = x;

~~y = y;~~

b = y; }

int main() {

// implicit call

Complex a(1, 6);

// explicit call

Complex b = Complex(5, 7);

a - printnumber();

b - printnumber(); return 0;

}

Constructor Overloading

Same like function overloading.

Just set the value of variables to 0 in case of multiple constructors.

Constructors with default arguments.

class Simple {

int data1, data2;

public:

Simple(int a, int b = 1) {

data1 = a;

data2 = b; }

};

→ like default arguments in functions.

Dynamic initialization of objects using constructors.

↓
Taking input from the user.

```
#include <iostream>
using namespace std;
```

```
class Bankdeposit {
```

```
    int principal, years;
```

```
    float interestrate, returnvalue;
```

```
public:
```

```
Bank Bankdeposit();
```

```
Bankdeposit
```

```
    Bankdeposit(int p, int y, int r); // r in fraction
```

```
    void show; };
```

we made this because even if we don't give the values of p, y, r, our object will be created.

```
Bankdeposit::Bankdeposit(int p, int y, int r) {
    principal = p; years = y; interestrate = r;
    returnvalue = principal;
```

```
    for (int i = 0; i < y; i++) {
        returnvalue = returnvalue * (1 + interestrate);
    }
}
```

```
Bankdeposit::Bankdeposit(int p, int y, int r) {
```

```
    interestrate = float(r)/100;
```

```
    };
```

```
void Bankdeposit::Show() {
```

```
    cout << "Principal amount was" << principal
```

```
    << ". Return value after" << years
```

```
    << "years is" << returnvalue << endl; }
```

```
int main() {
```

```
    Bankdeposit bd1, bd2, bd3;
```

```
    int p, y;
```

```
    float r;
```

```
    int R;
```

```
    cout << "Enter the values of p, y and r" << endl;
```

```
    cin >> p >> y >> r;
```

```
    bd1.Show(); bd1 = Bankdeposit(p, y, r);
```

```
    cout << "
```

```
    cin >> p >> y >> R;
```

```
    bd2 = Bankdeposit(p, y, R);
```

```
    bd2.Show();
```

```
    return 0; }
```

explicit constructor
calling

If you will be having multiple constructors in the program then don't forget to add even a default constructor.

Copy constructor

A constructor which makes a copy of another object.

When no copy constructor is found^{or defined} in the code, the compiler supplies its own default copy constructor to run.

A copy constructor is invoked when the object creation is created in the same line as the calling of the copy constructor else it'll not.

eg → Number z3 = z0;

```
#include <iostream>
```

```
using namespace std;
```

```
class Number {
```

```
    int a;
```

```
public:
```

```
    Number () {
```

```
        a = 0; }
```

```
    Number (int num) {
```

```
        a = num; }
```

```
    Number (Number &(obj)) {
```

```
        cout << "Copy constructor called" << endl;
```

```
        a = obj.a; }
```

```
    void display () { cout << "The number for two object is " << a << endl; }
```

```
};
```

default constructor.

copy constructor

formal object like formal argument.

Own reference object

```
int main() {
```

```
    Number x, y, z(45), z2;
```

```
    x.display();
```

```
    y.display();
```

```
    z.display();
```

→ actual object like actual argument. ~~if those~~
~~member~~ ~~is~~ ~~with~~
~~be~~ ~~called.~~

```
    Number z1(2); // copy constructor called.
```

```
    z1.display();
```

```
z2 = z; z2 = z; // copy constructor not called.
```

```
    z2.display();
```

// z1 should exactly resemble x, y or z:

```
    return 0; }
```

Destructors

↓ → free memory

takes no ~~value~~ arguments and return no values.

When the compiler gets to know that a particular object
is no more needed, the destructor is called.

```
#include <iostream>
```

```
using namespace std;
```

```
int count = 0; // global variable.
```

```
class num {
```

```
public:
```

```
    num() {
```

```
        count++;
```

```
        cout << "This is the time when constructor is called for object  
        number " << count << endl; }
```

Syntax for ~ class name() { }

~ num() {

cout << "This is the time when destructor is called for object number" << count << endl;

count--; }

int main() {

cout << "We are inside our main function" << endl;

cout << "Creating first object n1" << endl;

num n1;

{ cout << "Entering this block" << endl;

cout << "Creating 2 more objects" << endl;

num n2, n3; exiting

cout << "~~Eng~~ ~~Entering~~ this block" << endl;

}

cout << "Back to main" << endl;

return 0;

}

These objects will be in this block only

and not exist outside this block.

Inheritance

The act of inheriting class properties

The concept of reusability in C++ is supported using inheritance. we can reuse the properties of an existing class by inheriting it from it.



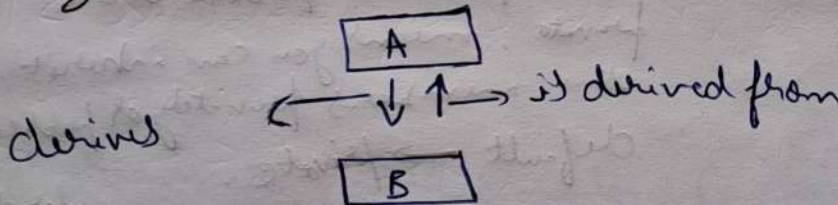
Existing class $\rightarrow$ Base class.

new inherited class $\rightarrow$ derived class.

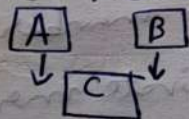
Forms of inheritance $\rightarrow$

- ① Multiple inheritance
- ② Hierarchical inheritance
- ③ Multilevel inheritance
- ④ Hybrid inheritance
- ⑤ Single inheritance.

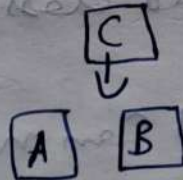
① Single inheritance $\rightarrow$ A derived class with only one base class.



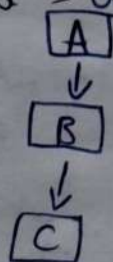
② Multiple inheritance $\rightarrow$ A derived class with more than one base classes



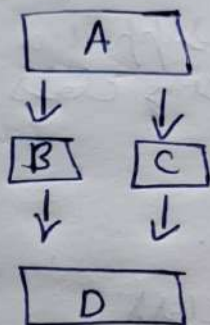
③ Hierarchical inheritance $\rightarrow$ Several derived classes from a single base class.



④ Multilevel inheritance $\rightarrow$ deriving a class from already derived class.



⑤ Hybrid inheritance → A combination of multiple and multilevel inheritances.



Syntax of inheritance

class derived class name : visibility mode base class name
{ }



Can be public or protected or private, means you can inherit the members privately or publicly or default → private, protectedly

Private visibility mode → The public members of the base class become the derived class' private members.

Public visibility mode → the derived class' public members.

Private members of the base class can never be inherited.



```
#include <iostream>
using namespace std;
// Base class
```

```
class Employee {
public:
    int id;
    float salary;
    Employee (int inpid)
    { id = inpid;
      salary = 34.0;
    }
}
```

Employee () {} → This default constructor has been called because the derived class 'objects' invoke the base class' default constructor, so, will not have an error.

// Derived class

```
class programmer: public Employee {
```

```
int id;
public:
    int language code;
    programmer (int inpid)
    { id = inpid;
      language code = 9;
    }
}
```

```
};
```

```
int main() {
```

```
    Employee herry (1), rohan (2);
    cout << herry.salary << endl;
    cout << rohan.salary << endl;
    programmer skillF (10);
```

```
    cout << skillF.language code << endl;
    cout << skillF.id << endl;
    return 0; }
```

Single inheritance

```
#include <iostream>
using namespace std;
class Base {
    int data_1; // this will not get inherited
```

```
public:
```

```
    int data_2;
```

```
    void setdata();
```

```
    int getdata1();
```

```
    int getdata2();
```

```
void Base::setdata() {
    data_1 = 10;
    data_2 = 20;
}
```

```
int Base::getdata1() {
    return data_1;
}
```

```
int Base::getdata2() {
    return data_2;
}
```

```
class Derived : public Base {
    int data_3;
```

```
public:
```

```
    void process();
```

```
    void display();
```

```
void Derived::process() {
    data_3 = data_2 *
        getdata1();
}
```

```
void Derived::display()
```

```
{ cout << "Value of data 1 is: " << getdata1();  
  cout << "Value of data 2 is: " << data2 << endl;  
  cout << "Value of data 3 is: " << data3 << endl;  
}
```

```
int main() {  
    Derived der;  
    der.setdata();  
    der.process();  
    der.display();  
    return 0;  
}
```

Protected Access Modifier

A access modifier which is like private but can be inherited.

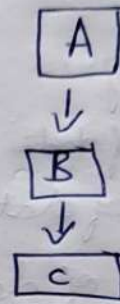
	public derivation	private derivation	protected derivation
① Private members	not inherited	not inherited	not inherited
② Protected members	protected	private	protected
③ Public members	public	private	protected

Multilevel inheritance

```
#include <iostream>
using namespace std;
```

The code is in my code editor.

No, it's below only.



Notes → ^① if we are inheriting B from A and C from B

① A is the base class for B and B is the base class for C.

② ~~A B~~ A → B → C is called the inheritance path.

Code →

```
#include <iostream>
using namespace std;
```

```
class Student {
```

```
protected:
```

```
rollnumber;
```

```
public:
```

```
void set_rollnumber(int r);
```

```
void get_rollnumber();
```

```
};
```

```
void Student::set_rollnumber(int r) {
    rollnumber = r; }
```

```
void Student::get_rollnumber() {
    cout << "Roll number is " << rollnumber << endl;
}
```

```
class exam: public Student
```

```
{ protected:
```

```
    float maths, physics;
```

```
public
```

```
    void set_marks( float, float);
```

```
    void get_marks();
```

```
};
```

```
void exam::set set_marks( float m1, float m2){
```

```
    maths = m1;
```

```
    physics = m2; }
```

```
void exam::get_marks1){
```

```
    cout << "marks in maths:" << maths << endl;
```

```
    cout << "marks in physics:" << physics << endl;
```

```
}
```

```
class result: public not exam {
```

```
    float percentage;
```

```
public:
```

```
    void display_result(){
```

```
        get_rollnumber();
```

```
        get_marks();
```

```
        cout << "Your result is" << (maths + physics) / 2 << "%" << endl;
```

```
    } };
```

```
int main(){
```

```
    result herry;
```

```
    herry.set_rollnumber(420);
```

```
    herry.set_marks( 90.0, 94.0);
```

```
    herry.display_results();
```

```
    return 0;
```

```
}
```

Multiple Inheritance

Syntax → derived class: visibility mode base class 1, visibility mode 1
{ };

```
Class Base1 {
```

```
protected:
```

```
int Base1int;
```

```
public:
```

```
void set base1int (int a) {  
    Base1int = a;  
}
```

```
}
```

```
Class Base2 {
```

```
protected:
```

```
int Base2int;
```

```
public:
```

```
void set base2int (int a) {  
    base2int = a;  
}
```

```
}
```

```
Class derived: public Base1, public Base2 {
```

```
public:
```

```
void Show () {
```

```
    cout << "The value of Base1 is" << Base1int << endl;
```

```
    Base2 - << Base2int << endl;
```

```
}
```

```
}
```

// protected members can be accessed directly within the derived classes that's why Base1int and Base2int didn't throw an error but you have to call

// call the function only outside the derived class.

```
int main() {
```

```
    derived hobby
```

```
    hobby.setsetBase1::int(25);
```

```
    hobby.setBase2::int(5);
```

```
    hobby.Show();
```

```
    return 0;
```

```
}
```

Ambiguity resolution in inheritance

arises when the base classes and the derived classes have the same named functions.

```
#include <iostream>
```

```
using namespace std;
```

```
class Base1 {
```

```
public: void greet() {
```

```
    cout << "Hello" << endl;
```

```
}
```

```
};
```

```
class Base2 {
```

```
public:
```

```
    void greet() {
```

```
        cout << "Hi" << endl;
```

```
    }
```

```
class derived: public Base1, public Base2 {
```

```
    int a;
```

```
public:
```

```
    void greet() {
```

```
        Base2::greet();
```

```
    }
```

→ This line tells the compiler whose greet() function to use.

```
int main() {
    Derived d;
    derived
    d.greet();
    return 0;
}
```

output → Hi

Example 2:

```
class B {
    public:
    void say() {
        cout << "Hello" << endl;
    }
};
```

```
class D : public B {
```

```
    public:
    void say() {
```

```
        cout << "Hi" << endl;
    }
};
```

→ derived class' say() method will override the base class' say() method.

This ambiguity gets resolved on its own.

```
int main() {
```

```
    B b;
```

```
    b.say();
```

```
    D d;
```

```
    d.say();
```

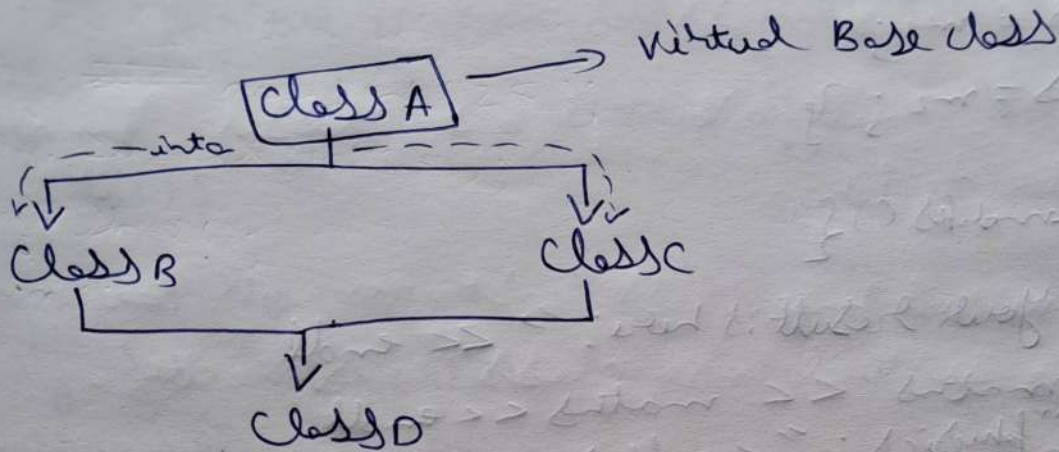
```
    return 0;
```

```
}
```

Output → Hello
Hi

Virtual Base class

Used to resolve duplicated data member inheritance.



a will be inherited only once in D because of the virtual base class.

Syntax → `Class B: virtual public A {`
`——— }`

`Class C: virtual public A {`
`——— }`

`public virtual = virtual public`

eg → `#include <iostream>`
`using namespace std;`

`class Student {`

`protected:`

`int roll no;`

`public:`

`void set number (int a) {`
`roll no = a;`

`}`

`void print number () {`
`cout << "roll no" << roll no << endl;`
`} };`

`class test : public virtual Student {`

`protected:`

`float marks, physics;`

public: float m1, float m2

void set marks () {

maths = m1;

physics = m2; }

void print marks () {

cout << "your result is here: " << endl

<< "maths" << maths << endl

<< "physics: " << physics << endl;

}

};

class Sports {^{virtual} : public Student {

protected:

float score;

public:

void set score (float scores) {

score = scores;

}

void print score () {

cout << "your score is: " << score << endl;

};

class result: public test, public Sports {

private:

float total;

public:

void display () {

total = maths + physics + score;

print-number ();

Only 1 copy of roll number
will be inherited in result
instead of 2.

That's the purpose of
virtual base classes.

```
print marks();
```

```
print score();
```

```
cout << "Your total is: " << total << endl; } };
```

~~data~~

```
int main() {
```

```
    student hobby;
```

```
    hobby.set number (420);
```

```
    hobby.set marks (99.5, 99.7);
```

```
    hobby.set score (7);
```

```
    hobby.display();
```

```
    return 0;
```

```
}
```

Constructors in derived classes

- ① If the base class' constructor doesn't have any arguments, there is no need of any constructor in the derived class, it'll use the base class' constructor.
- ② If the Base class constructor has arguments then you have to pass the values for that constructor in the ~~base~~ ^{derived} class.
- ③ If both Base and derived classes have constructors, base class' constructor is executed first.

Constructors in multiple inheritance.

base classes are constructed in the order they are written during the class declaration.

In multilevel inheritance, the constructors are executed in the order of the inheritance.

Special case for virtual Base class →

- ① The constructors for virtual base classes are called before a non virtual base class.
- ② If there are multiple virtual base classes, they are invoked in the order declared.
- ③ Any non virtual base class constructor will be invoked after the virtual base class' constructor.

Code example →

Special Syntax → The constructor of the derived class receives all the arguments at once and then will pass to the respective base classes.

Derived-Constructor (arg₁, arg₂, arg₋₋₋): Base1-Constructor (arg₁, arg₂), Base2-Constructor (arg₃, arg₄)
{
--- } Base1-Constructor (arg₁, arg₂)

Constructor in derived class Code example →

```
#include <iostream>
using namespace std;
```

```
class Base1 {
    int data1;
public:
    Base1 Base1(int i) {
        data1 = i;
        cout << "Base1 class constructor called" << endl;
    }
    void printdata1() { cout << "The value of data1 is" << endl; }
};
```

// Same for Base2, just swap Base2 with Base1

```
class Derived : public Base2, public Base1 {
    int derived1, derived2;
public:
    → That Special syntax.
    Derived(int a, int b, int c, int d) : Base2(b), Base1(a) {
        derived1 = c; derived2 = d;
        cout << "Derived class constructor called" << endl;
    }
    void printdataDerived() {
        cout << "data1 = " << derived1 << endl;
        cout << "derived2 = " << derived2 << endl;
    }
};
```

```
int main() {
    Derived hobby(1, 2, 3, 4);
    hobby.printdata1();
    hobby.printdata2();
    hobby.printdataDerived();
    return 0;
}
```

initialisation list in constructors.

Syntax for initialisation list $\rightarrow$

Constructor (arguments) : initialisation-section {
// code }

dynamic allocation.

```
class test {
```

```
    int a, b;
```

```
public:
```

```
    test(int i, int j) : a(i), b(j) {
```

```
        cout << "Constructor executed" << endl;
```

```
        cout << "a = " << a << endl;
```

```
        cout << "b = " << b << endl;
```

```
    }
```

$\rightarrow$ we can write arithmetic expressions too here.

```
int main()
```

```
{ test t(4, 6);
```

```
    return 0;
```

```
}
```

Output

Constructor executed

a = 4

b = 6

```
test(int i, int j) : b(j), a(i+b)  $\rightarrow$ 
```

This will throw an error as a is initialized first.

Not throwing an error but it will produce a garbage value. (a)

```
test(int i, int j) : a(i), b(a+j)  $\rightarrow$  This will run flawlessly.
```

New and delete operators in context of pointers.

```
#include <iostream>
using namespace std;
```

```
int main() {
    // Basic eg
    int a = 4;
    int *ptr = &a;
    cout << "The value of a is" << *(ptr) << endl;
    return 0; }
```

// Using new keyword

Boiler plate code

```
int main() {
    float *p = new float(40.78);
    cout << "The value at address p is" << *(p) << endl;
    return 0; }
```

we didn't have to create an additional float variable, it got dynamically created.

This will print 40.78

// eg 2 → Boiler plate code

```
int main()
int *arr = new int[3];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
```

```
cout << "The value at arr[0] is" << arr[0] << endl;
```

```
return 0;
}
```

you can also write
*(arr+1)
*(arr+2)

delete[];
↓
deletes the array
and frees the
memory space.

Pointers to Objects and Arrow functions.

```
#include <iostream>
using namespace std;
```

```
class complex {
    int real, imaginary;
public:
```

```
    void getdata() {
```

```
        cout << "The real part" << real << endl;
```

```
        cout << "The imaginary part" << imaginary << endl;
    }
```

```
    void setdata(int a, int b) {
```

```
        real = a; imaginary = b;
    }
```

```
int main() {
```

```
    // complex c1;
```

```
    // complex * ptr = &c1;
```

```
    complex * ptr = new complex;
```

→ different ways of creating an object.

```
    // (*ptr).setdata(1,4);
```

↳ the parenthesis is important to set the precedence as dot operator has higher precedence as compared to *.

```
    ptr → setdata(1,4);
```

→ This means using this function on the ~~value~~ address to which the pointer is pointing.

```
    return 0; }
```

Array of objects using pointers.

```
#include <iostream>
using namespace std;
```

```
class Shopitem {
```

```
    int id;
```

```
    float price;
```

```
public:
```

```
    void setdata(int a, float b) {
        id = a;
        price = b;
    }
```

```
    void getdata() {
        cout << "id" << id << endl;
        cout << "price" << price << endl;
    }
```

```
example → int main() {
    int size = 3;
```

```
    Shopitem* ptr = new Shopitem[size];
```

```
    Shopitem* ptrtemp = ptr;
```

```
    int p, i;
```

```
    float q;
```

```
    for (i = 0; i < size; i++)
```

```
    { cout << "Enter item id and price of " << i+1 << endl;
      cin >> p >> q;
```

```
      ptr → setdata(p, q);
```

```
      ptr++; }
```

```
    for (i = 0; i < size; i++) {
```

```
        cout << "Item number: " << i+1 << endl;
        ptrtemp → getdata();
```

```
        ptrtemp++; }
```

```
    return 0;
```

```
}
```

In the code, a ^{copy} ~~copy~~ of the pointer has been created because, when the last iteration of the setData function gets completed, the pointer (original) would ^{be} ~~have been~~ pointing ~~to~~ a completely irrelevant location.

So, to use the ~~for~~ setData function in the next loop, the ptrtemp has been created.

This pointer

this is a keyword which is a pointer which points to the object which invokes the member function.

```
class a {
```

```
int b;
```

```
public:
```

```
void setData (int b) {
```

```
    this → b = b; }
```

```
void getData ( ) {
```

```
    cout << "The value of a is" << b << endl;
```

```
} }
```

Another use case →

```
class a { int b;
```

```
public:
```

```
    a & setData (int b) {
```

```
        this → b = b;
```

```
        return *this; }
```

```
int main ( ) {
```

```
    a c (4)
```

```
    a c;
```

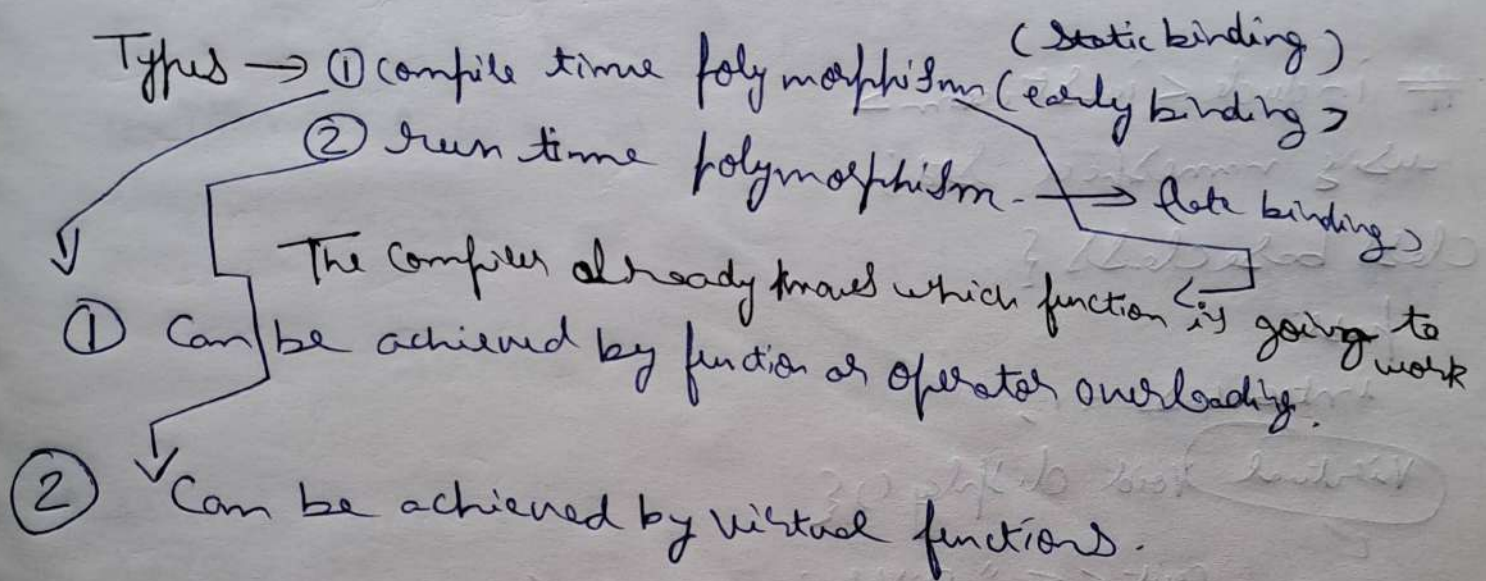
```
    c.setData(4).getData();
```

both functions can be used together.

Polymorphism

one name, Multiple forms

eg → function overloading, Operator overloading, Virtual functions



Pointers to derived class.

If you have made a pointer from the base class ^{to} point to derived class object, it will not throw an error but a function like display() will run of the base class only as the pointer ultimately is pointing to a base class object only.

You can only access those functions or properties that you have inherited from the base class to the derived class using the base class pointer pointing to a derived class object.

Virtual functions.

Used to tell the compiler to run the function of the derived class only, overriding the default behaviour of the base class pointers.

```
#include <iostream>
using namespace std;

class baseclass {
public:
    int varbase = 1;
    virtual void display() {
        cout << "Displaying varbase" << varbase << endl;
    }
};
```

important: virtual

```
class derivedclass : public baseclass {
public:
    int varderived = 2;
    void display() {
        cout << "Displaying varbase varderived" << varderived << endl;
    }
};
```

```
int main() {
    baseclass * baseclassptr;
    baseclass objbase;
    derivedclass objderived;
    baseclassptr = &objderived;
    baseclassptr -> display(); return 0;
}

// like int* ptr;
// like ptr = &i;
```

output -> Displaying varderived 2

Rules for virtual functions :

- ① They can't be static.
- ② Can be accessed by object pointers.
- ③ They can be a friend of another class.
- ④ a function in the base class might not be used at all.
- ⑤ A virtual function is ~~defined~~ ^{defined} in the base class then it is not necessary to redefine it in the derived class.

Abstract base class and pure virtual functions.

Abstract base class → it is made to use its properties in the derived classes. And not use it itself.

pure virtual or do nothing function → ^{Syntax} virtual datatype function name() = 0
↓
do nothing function.

Try - Catch block

Syntax → ~~try~~ try { // put the risky code that might throw an error. | exception }

catch (exception type e) {

// code to handle the exception
}

include <stdexcept>

eg → try {

int amount;

cout << "Enter deposit amount:";

cin >> amount;

if (amt <= 0) {

~~cout << "Invalid deposit Amount: endl;~~

}

→ throw invalid_argument

(... Invalid deposit amount)

→ inbuilt class

in std except.

catch (exception e) {
e.what(); }

throw runtime_error();

File I/O

File input output

#include <fstream>

useful classes for working with files in C++.

① fstreambase

② ifstream

③ ofstream

derived from fstreambase.

included in the header file

eg → #include <string>
#include <iostream>
#include <fstream>

using namespace std;

int main() {

string st = "Halcy";

string st2;

// Opening and ^{writing} ~~reading~~ files using constructors

ofstream out ("Sample.txt");

out << st;

Object name, can be different.

// Opening and reading files ~~also~~ using constructors.

ifstream in ("Sample2.txt");

in >> st2;

getline (in, st2); → ~~it~~ reads only till before a space or a new word.

↓ ↓
Object ~~file~~ ^{string} name. → whole line is read.

cout << st2;

return 0;

File I/O [Reading and writing a file]

`Object.close();` → Closes the file.

`Object.open();` → Opens the file.

`Object.eof()` → End of file.

eg →

```
int main() {  
    while (Object.eof() == 0) {  
        getline(Object, string);  
        cout << string << endl; } Object.close();  
    return 0; }
```

Templates Parameterized Classes

Syntax $\rightarrow$ `template < class T >`

`class Vector {`

`T* arr;`

`public:`

`Vector();`

`};`

Templates are to classes, what
classes are to objects.

T can be float, int, char etc.

`int main() {`

`Vector<int> myVec(ptr); }`

eg $\rightarrow$ `#include <iostream>`

`using namespace std;`

`template < class T1, class T2 >`

`class myclass {`

`public:`

`data < T1 data1;`

`types < T2 data2;`

`myclass(T1 a, T2 b) {`

`data1 = a; data2 = b; }`

`void display() {`

`cout << this->data1 << " " << this->data2; }`

`int main() { myclass<int, char> obj(1, 'c');`

`obj.display();`

`return 0;`

output $\rightarrow$ 1 c

Templates with default parameters
like default functions.

```
#include <iostream>
```

```
using namespace std;
```

```
template < class T1 = int, class T2 = float, class T3 = char >
```

```
class Haldy {
```

```
public:
```

```
T1 a;
```

```
T2 b;
```

```
T3 c;
```

```
Haldy(T1 x, T2 y, T3 z) {  
    a = x; b = y; c = z; }
```

```
void display() {
```

```
    cout << "The value of a = " << a << endl;
```

```
    cout << "          " << b << endl;
```

```
    cout << "          " << c << endl;
```

```
};
```

```
int main() { Haldy < > h(4, 6.4, 'c')  
             // leave it empty
```

```
             h.display();
```

```
             return 0; }
```

Function templates

```
#include <iostream>
```

```
using namespace std;
```

```
template < class T1, class T2 >
```

```
float funcAvg funcAverage (T1 a, T2 b) {  
    float avg avg = (a+b)/2.0;  
    return avg avg; }
```

```
int main() {
```

```
    float a;
```

```
    a = funcAverage(5, 2);
```

```
    cout << "The average of the numbers is" << a << endl;  
    return 0; }
```

Overloading Templates →

```
#include <iostream> using namespace std;
```

```
void func(int a) {
```

```
    cout << "I'm first func" << a << endl; }
```

```
template < class T >
```

```
void func(T a) {
```

```
    cout << "I'm templated func" << a << endl; }
```

```
int main() {
```

```
    func(4);
```

```
    return 0;
```

```
}
```

// Exact match takes the highest priority
so instead of the templated - func1
will be called.

STL

Standard template library

Components of STL →

- ① Containers → Store data use template classes
- ② Algorithms → use template functions
- ③ Iterators → object pointing to an element in a container
↓
Connect algorithms with containers.
→ ~~handle~~ handled just like pointers.

Containers →

- ① Sequence Containers → Store data in linear fashion
- ② Associative containers → Direct access
- ③ Derived Containers → Real world modelling.

When to use which →

Sequence →

① Vector → Random access — fast

② list → Random access → Slow.

Associative → All Operators are fast except Random access.

③ Derived Containers → depends on data structures.

Vector

→ works similar to an array but you can add or delete elements faster and unlike arrays you can add as many elements as you want.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

refer C++ reference

List, STL maps or function Objects/ functors

refer C++ reference.

Standard Template Library

#include <bits/stdc++.h>, to include all the libraries at once.

Operator Overloading

It is a type of compile time polymorphism giving special meaning to an already existing operator without changing its meaning.

Code example →

```
#include <iostream> using namespace std;
```

```
class Number {
```

```
    int value;
```

```
    public:
```

```
        Number(int v=0) : value(v) {}
```

```
    // overload + operator
```

```
        Number operator + (const Number & other) const {  
            return Number(value + other.value);  
        }
```

```
        void display() {
```

```
            cout << "value" << endl;
```

```
        };
```

```
int main() {
```

```
    Number N1(5);
```

```
    Number N2(10);
```

```
    Number N3 = N1 + N2;
```

```
    N3.display();
```

```
    return 0;  
}
```

output = 15

Operator overloading

To assign more operations in one operator.

function $\rightarrow$ Operator ();

Syntax $\rightarrow$ return type Operator Op (argument list)
 $\{$ "body"; $\}$

$\left. \begin{array}{l} \cdot \\ \cdot \\ \cdot \\ ? \\ \cdot \\ \text{sizeof}() \end{array} \right\} \rightarrow$ these operators can be overloaded.